201CP: OBJECT ORIENTED PROGRAMMING WITH C++

Prof. Tirth Patel

Computer Engineering Dept.

BVM Engineering College

Procedure-Oriented Programming

- **Procedure-Oriented Programming (POP)** is a programming paradigm that is **focused on functions** (also called procedures or routines). It is one of the **earliest and traditional styles** of programming, especially used in languages like **C**, **Pascal**, and **Fortran**.

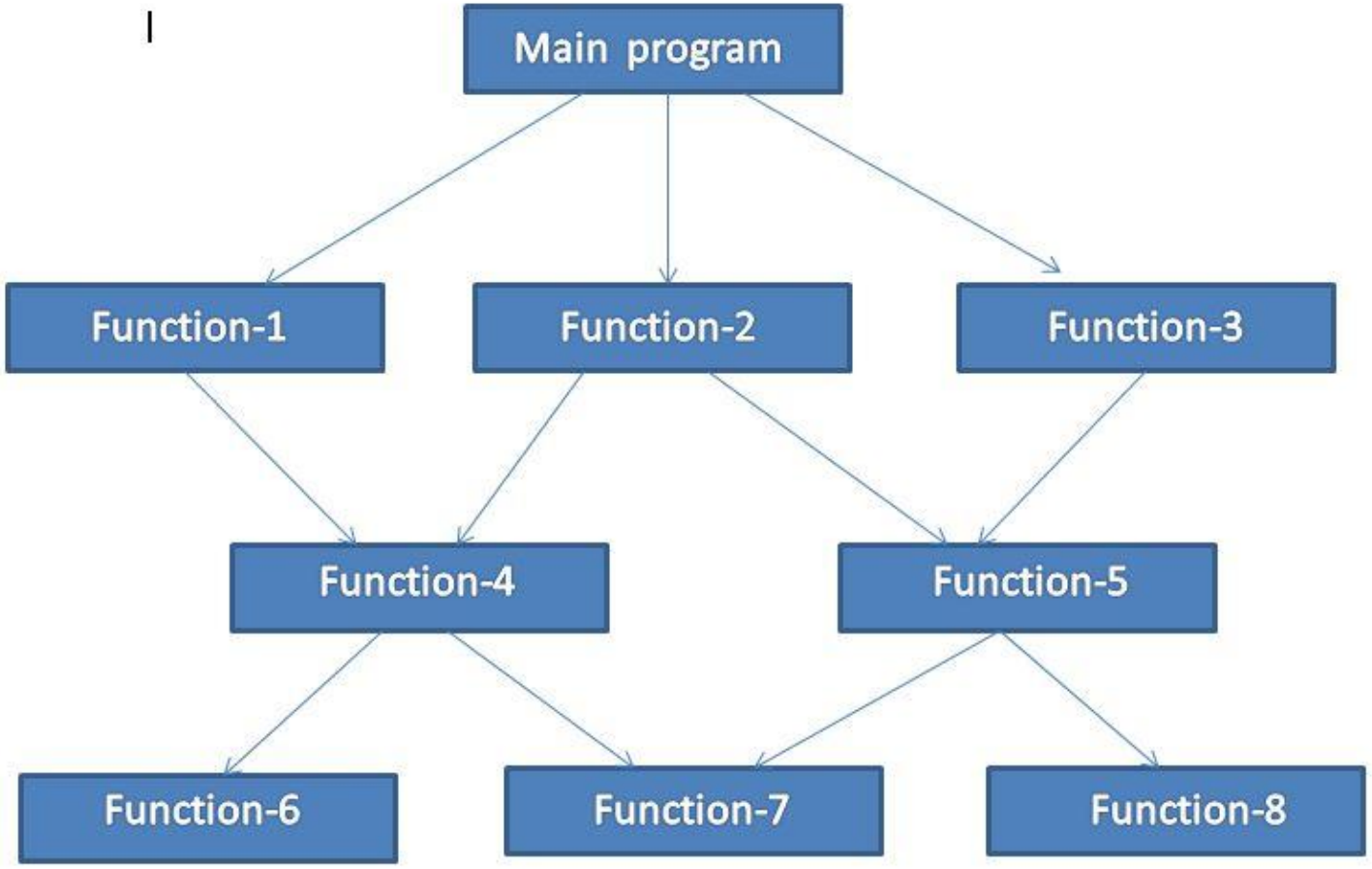
- **Key Characteristics of POP:**

1. Function-Centric:

- The main program is divided into smaller functions.
- These functions carry out specific tasks.
- Example: In C, you write void display() or int add(int a, int b).

2. Top-Down Approach:

- The design starts from the main logic and breaks down into smaller functions.
- Focus is on "what needs to be done" rather than "who is doing it".



Structure of procedure oriented programming

Contd.

3. Global Data Sharing:

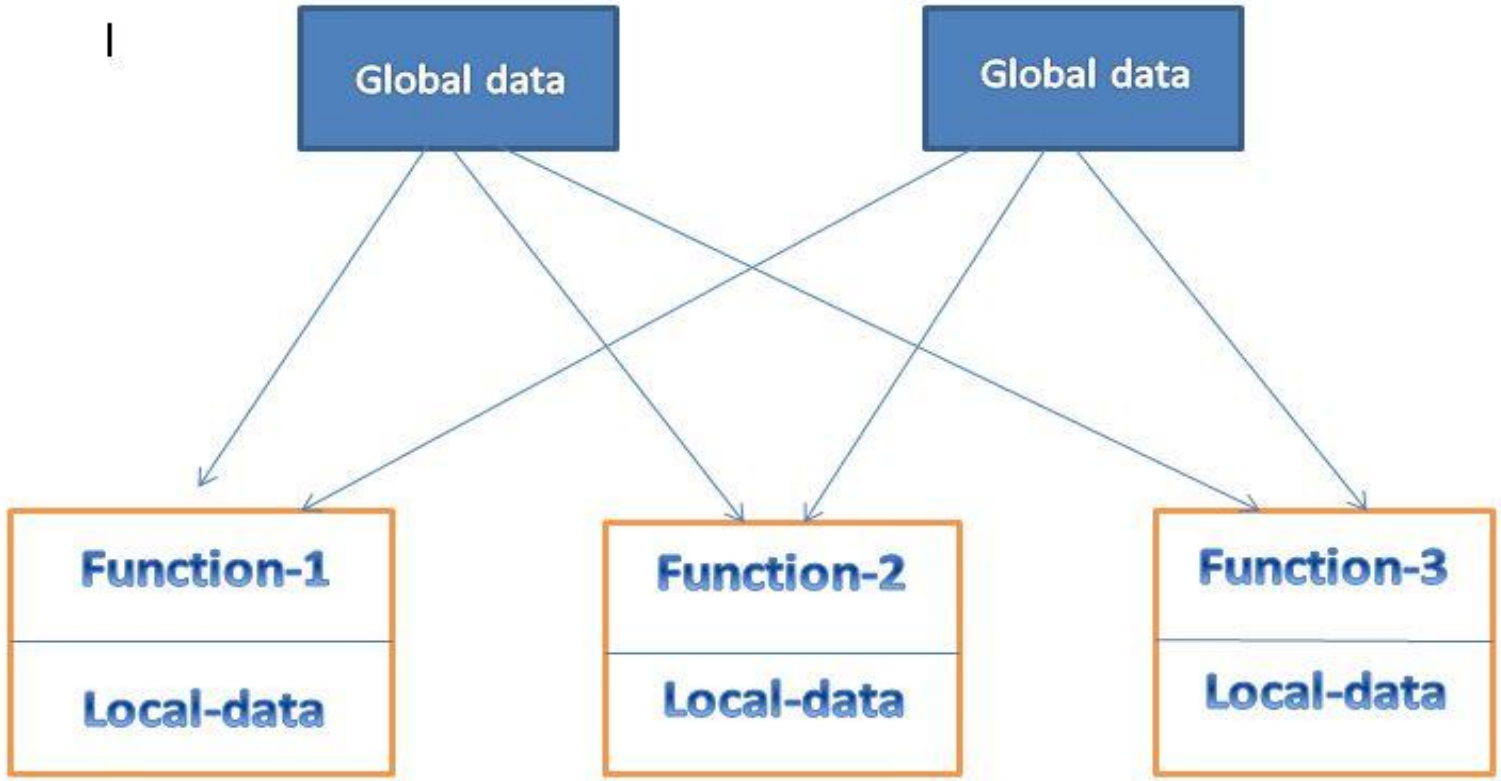
- Data is usually shared using global variables across functions.
- Any function can modify global data, which can lead to bugs

4. Reusability Limited:

- Functions can be reused, but there's no concept of data encapsulation or inheritance.

5. No Real-World Mapping:

- It doesn’t model real-world entities (like a "Student" or "Employee") directly.
- The focus is more on operations rather than objects



Relationship of data and function in procedural-programming

Contd.

- **Limitation**
- When we revise an external data structure we also need to revise all functions that access the data
- Does not model real world problems very well

Object-Oriented Programming

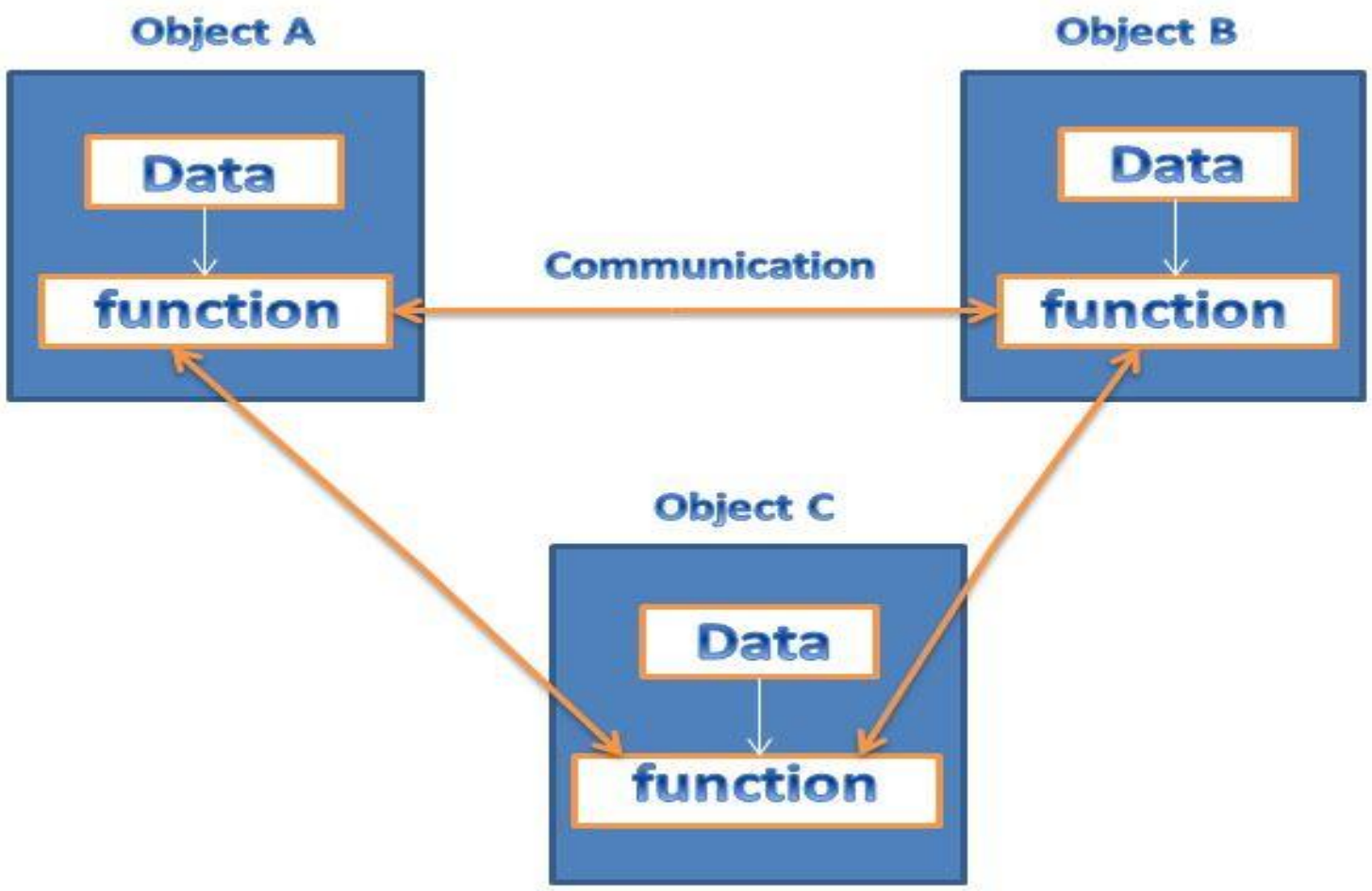
- **Object-Oriented Programming** is a programming paradigm that is centered around the concept of **"objects"**, which are instances of **classes**. It focuses more on **data (objects)** and **how to interact with it**, rather than just functions and logic.

- ❑ It treats data as a critical element
- ❑ It tries data more closely to the functions that operate on it
- ❑ Data and functions together known as object

characteristics

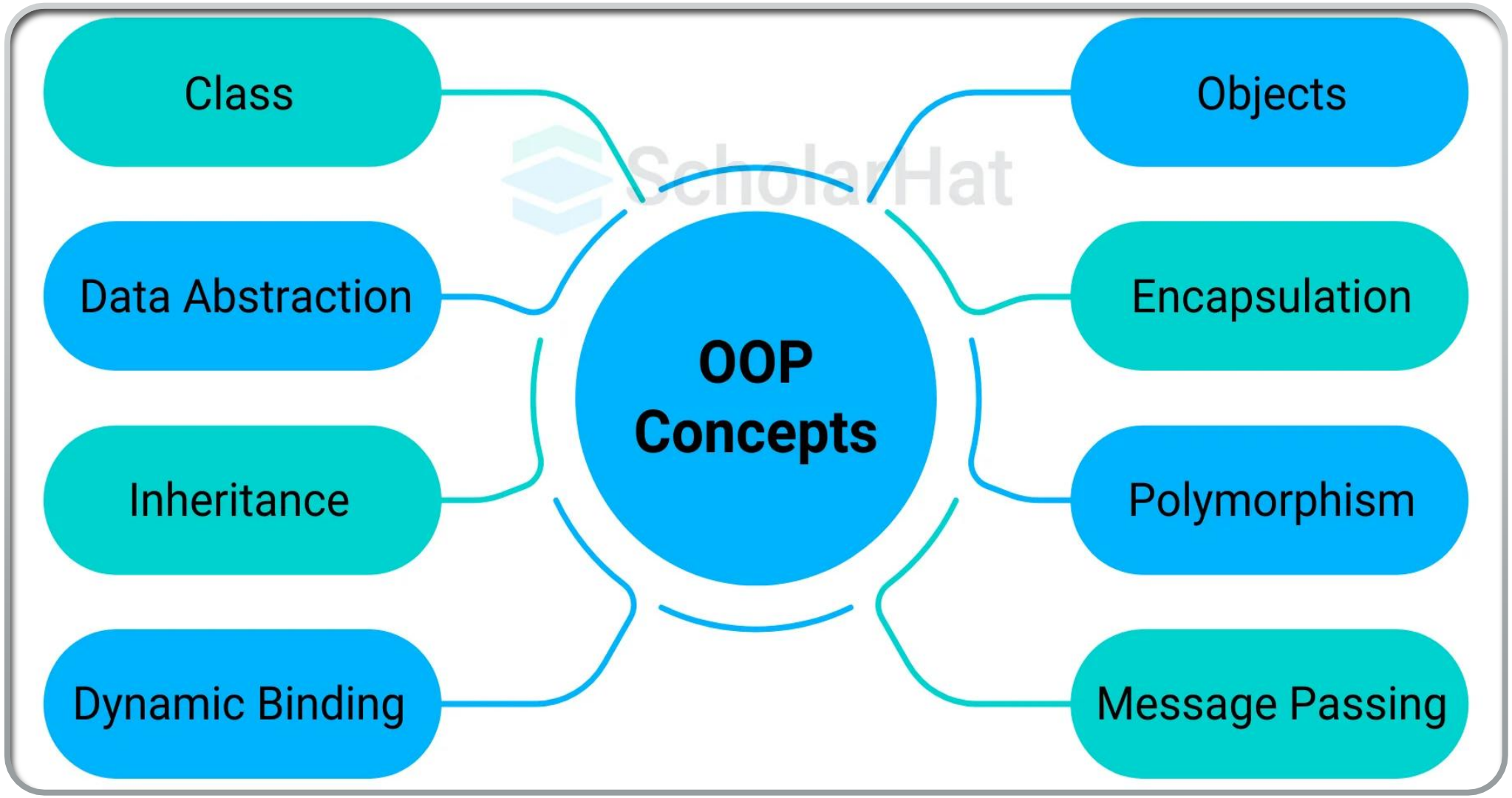
- ❖ Emphasis on data rather than function
- ❖ Programs are divided into objects
- ❖ Data is hidden and cannot be accessed by external functions
- ❖ Objects may communicate with each other through functions
- ❖ Follows bottom-up approach

Object-Oriented Programming



ORGANIZATION OF DATA AND FUNCTIONS IN OOPS

Basic concepts of object-oriented programming



Contd.

- **Simple Analogy:**

Imagine you are running a hospital,

- In **Procedure-Oriented Programming**, you focus on **how the hospital runs** — the procedures like admitting patients, assigning doctors, billing, etc. Everything runs through a **step-by-step process**, and the **data (patients, doctors)** is passed between these procedures.
- In **Object-Oriented Programming**, you focus on the **people and things** — like **patients, doctors, nurses, rooms** — and each of them **knows how to take care of itself** (data + actions). You're more focused on **modeling the real-world entities**.

POP = Process is the hero, data just follows.

OOP = Data is the hero, process is hidden inside the object.

Benefits of OOP

Benefit	Explanation
1. Modularity	Code is organized into classes and objects, making it modular and easier to manage.
2. Reusability	Classes and objects can be reused across projects using inheritance and composition.
3. Maintainability	Encapsulation allows easy modification of code without affecting other parts.
4. Security (Data Hiding)	Data can be kept private in a class using access specifiers like private.
5. Real-World Mapping	Programs reflect real-world entities more naturally (e.g., Student, Car, BankAccount).
6. Scalability	OOP makes it easier to scale applications by adding new features without rewriting large codebases.
7. Extensibility	Code can be extended by adding new classes or functions without touching existing ones.
8. Polymorphism	Enables the same interface to behave differently depending on the object — simplifies code.

Applications of OOP

Domain	Example Applications
1. Software Development	Operating systems, compilers, database management systems (e.g., Windows OS, MySQL engine)
2. Web Development	Frameworks like Django (Python), Laravel (PHP), ASP.NET (C#) use OOP principles.
3. Mobile App Development	Android (Java/Kotlin), iOS (Swift/Objective-C) are fully OOP-based.
4. Game Development	Unity (C#) and Unreal Engine (C++) use OOP for game objects, physics, and UI.
5. GUI Applications	Desktop applications like text editors, IDEs, and browsers use OOP for handling windows, buttons, etc.
6. Embedded Systems	Classes and encapsulation help manage hardware modules (e.g., motor control, sensors).
7. Artificial Intelligence & Machine Learning	Libraries like TensorFlow, PyTorch use OOP to structure models and data pipelines.
8. Financial and Banking Systems	Account, transaction, customer—everything is modeled using classes.

A SIMPLE C++ PROGRAM

```
#include <iostream>          // 1. Preprocessor Directive
using namespace std;         // 2. Namespace Declaration
  int main() {                 // 3. Main Function
      cout << "Hello, World!";  // 4. Statement
      return 0;                // 5. Return Statement
}
```

Contd.

- **#include <iostream>** tells the compiler to include the standard input/output library.

(https://cppreference.com/index.html :- comprehensive and trusted reference site for C and C++ language and standard)

- A **namespace** is a **container** in C++ that allows you to group **variables, functions, classes**, etc., under a **unique name.**
  It helps in **avoiding name conflicts** when your code uses libraries or other people's code that may have the same function or variable names.

Example: Without namespace

```
int value = 10;

   int main() {
    cout << value;  // Which 'value'? Could cause conflict
if 'value' is declared in another library.
}
```

Problem: If another header file or library also has a value, it will confuse the compiler. This is where namespace helps.

Contd.

Example: Using Namespace

```
#include <iostream>
using namespace std;  // Tells compiler to use the "std" namespace

  int main() {
    cout << "Hello, World!";  // Actually std::cout
    return 0;
}
```

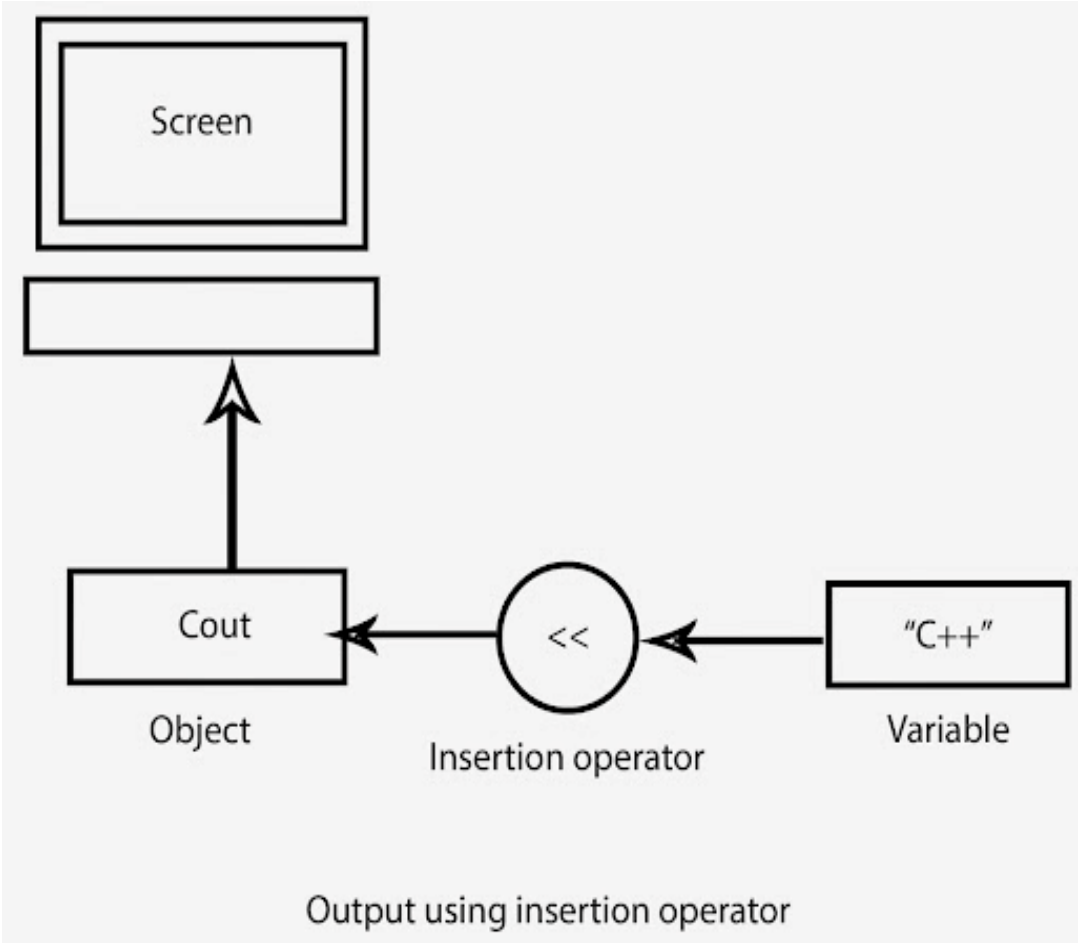
- **std** stands for Standard, and it's the namespace for all C++ standard library features like cin, cout, string, vector, etc.

  - `std::cout << "Hello"; //is the fully qualified name`

  - ```
    using namespace std;
    cout << "Hello";
    //is a shortcut to avoid writing std:: every time.
    ```
    ```

Output Operator

- In C++, the **output operator** is **<<**
- It is called the **insertion operator** and is used with **cout** to display output to the screen.
- **Basic Syntax:** cout << "Text to print";
- cout comes from the standard output stream and is part of the std namespace.
- Type-safe (automatic type handling, format specifiers not needed).



Output using insertion operator

Contd.

Examples:

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, World!";
    return 0;}
```

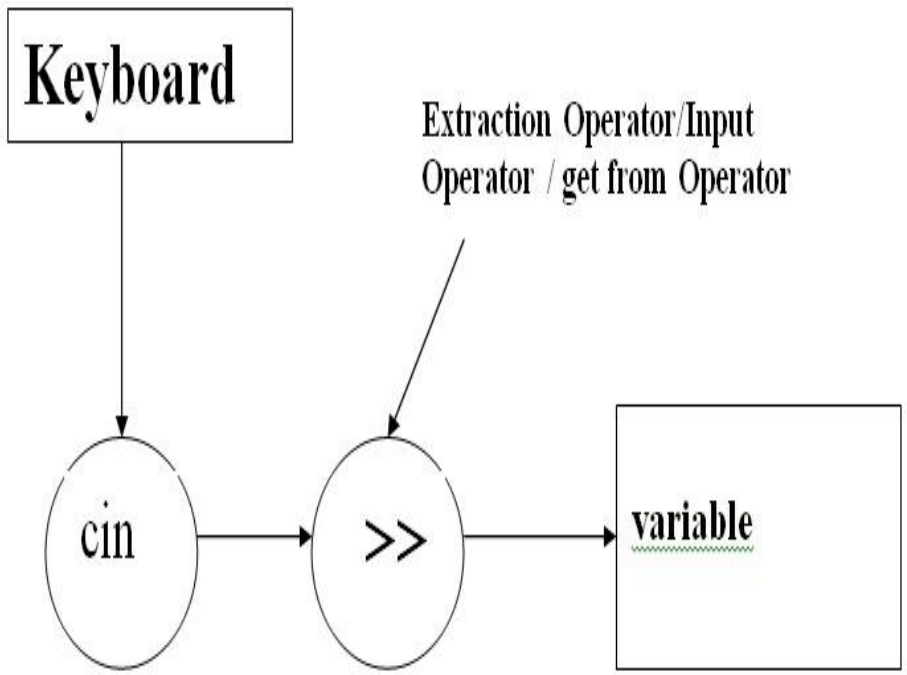
```
int a = 10;
cout << "Value of a: " << a << endl;
```

Output:
Value of a: 10

You can output:
Strings: cout << "Name";
Integers: cout << 5;
Characters: cout << 'A';
Variables: cout << age;

Input Operator

- In C++, the **Input operator** is **>>**
- The operator >> is known as **extraction** or **get from operator**. It extracts (or takes) the value from the keyboard and assigns it to the variable on its right and is used with **cin**
- **Basic Syntax:** cout >> number1 ;



Contd.

Examples:

```
#include <iostream>
using namespace std;

int main() {
    int age;
    cout << "Enter your age: ";
    cin >> age;
    cout << "Your age is: " << age;
    return 0;
}
```

Output:

Enter your age: 20
Your age is: 20

Contd.

❖ **Note on Strings:**

- Accepts only one word.
- For full line input need to use **getline(cin, name);**

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string name,fname;
    cout << "Enter the Full Name: ";
    cin >> name;
    cout << name<<endl;

    return 0;}
```

Output:

```
Enter the Full Name: Daksh Patel
Daksh
```

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string name;
    cout << "Enter the Full Name: ";
    getline(cin, name);
    cout << name ;

    return 0;
}
```

Output:

```
Enter the Full Name: Daksh Patel
Daksh Patel
```

cascading of I/O Operators

- It refers to using multiple input (>>) or output (<<) operators in a single statement in C++.
- This makes code more concise and readable when handling multiple inputs or outputs.

```
// Cascading Output (<<) Operators
#include <iostream>
using namespace std;

int main() {
    int a = 5, b = 10;
    cout << "The value of a is: "
<< a << " and b is: " << b <<
endl;
    return 0;
}

Output:
The value of a is: 5 and b is: 10
```

```
// Cascading Input (>>) Operators
#include <iostream>
using namespace std;

int main() {
    int x, y;
    cout << "Enter two numbers: ";
    cin >> x >> y;
    cout << "You entered: " << x
<< " and " << y << endl;
    return 0;
}

Output:
Enter two number: 7 8
Your entered: 7 and 8
```

Manipulators

- Manipulators are operators that are used to format the data display. The most commonly used manipulators are endl and setw. This makes code more concise and readable when handling multiple inputs or outputs.
- Commonly Used Manipulators:

Manipulator	Description	Header File
endl	Inserts a new line and flushes the output buffer	<iostream>
setw(n)	Sets field width to n (for next output only)	<iomanip>
setfill(c)	Fills empty space with character c	<iomanip>
setprecision(n)	Sets decimal precision for floating-point numbers	<iomanip>
left / right	Aligns output to left or right within width	<iomanip>
fixed	Outputs floating-point numbers in fixed-point notation	<iomanip>
showpoint	Forces decimal point for float values (like 5.0)	<iomanip>
hex, oct, dec	Changes number base (hexadecimal, octal, decimal)	<iostream>

Contd.

- Example:

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    int rollNo = 101;
    string name = “Krish";
    float marks = 92.5678;

    cout << "Student Report" << endl;  // 1. endl

    // 2. setw(), 3. setfill()
    cout << setfill('-') << setw(30) << "-" << endl;

    cout << left << setw(10) << "Roll No" << ": " << rollNo << endl;
    cout << left << setw(10) << "Name"    << ": " << name << endl;

    // 4. setprecision()
    cout << left << setw(10) << "Marks"   << ": "
         << fixed << setprecision(2) << marks << endl;

    cout << setfill('-') << setw(30) << "-" << endl;

    return 0;
}
```

Output:-

```
Student Report
------------------------------
Roll No   : 101
Name      : Krish
Marks     : 92.57
------------------------------
```

Reference Variable

- A reference variable in C++ is an **alias** (another name) for an existing variable. Once a reference is initialized with a variable, it can be used as an alternate name for that variable.
- **Syntax :** **data-type & reference-name = variable-name**
- Example:

float total = 100;

float & sum = total;

- total is a **float type** variable that has already been declared; sum is the alternative name declared to represent the variable total. Both the variables refer to the same data object in the memory.

cout << total ;

and

Cout << sum ;

Both print the value **100**

If , sum = 0 ;

Will change the value of both the variable to Zero.

Contd.

- A reference variable in C++ is an **alias** (another name) for an existing variable. Once a reference is initialized with a variable, it can be used as an alternate name for that variable.
- **Syntax :** **data-type & reference-name = variable-name**
- A reference variable must be initialized at the time of declaration.

Example:

```
float total  = 100;

float & sum = total;
```

- total is a **float type** variable that has already been declared; sum is the alternative name declared to represent the variable total. Both the variables refer to the same data object in the memory.

```
cout << total ;
```

and

```
Cout << sum ;
```

Both print the value **100**

If , sum = 0 ;

Will change the value of both the variable to Zero.

Example:

```
int n[10];
int & x = n[3]; //x is alias for n[3]

int x;
int *p = &x;
int & m = *p;
//The set of declarations causes m to refer to x which is pointed to by the pointer p.
```

major application of reference variables is in passing arguments to functions. Consider the following:

```
void square(int &x)  // uses reference
{
    x = x * x;      // x is incremented; so also n
}

   int main() {
    int n = 5;
    square(n);
    cout << n;  // Output: 25
}
```

When the function call square(n) is executed, the following initialization occurs: int & x = n;

thus , x becomes alias of n after the executing the statement square(n) ;

such function calls are known as call by reference.

PROGRAMMING EXERCISES

1. Write a program to display the following output using a single cout statement.

```
Maths = 90

Physics = 77
Chemistry = 69
```

2. Write a C++ program that will ask for a temperature in Fahrenheit and display it in Celsius.

3. Write a function using reference variables as arguments to swap the values of a pair of integer

Scope Resolution Operator

- The scope resolution operator **::** in C++ is used to access

  global variable,

  define a function outside a class,

  static member of a class,

  Accessing Namespaces.

- It helps to specify the context (scope) in which an identifier (like a variable or function) is defined.

Contd.

Example: Accessing Global Variables When Local Variable Shadows It

```
#include<iostream>
using namespace std;

int x = 10; // Global variable

int main() {
    int x = 20; // Local variable
    cout << "Local x = " << x << endl;        // 20
    cout << "Global x = " << ::x << endl;     // 10
    return 0;
}
```

Contd.

Example: Defining Class Member Functions Outside the Class

```
#include<iostream>
using namespace std;

class MyClass {
public:
    void show();
};

// Using scope resolution operator to define function outside the class
void MyClass::show() {
    cout << "Hello from MyClass!" << endl;
}

int main() {
    MyClass obj;
    obj.show();
    return 0;}
```

Contd.

Example: Accessing Static Members of a Class

```
#include<iostream>
using namespace std;

class Counter {
public:
    static int count;
};

int Counter::count = 0; // Define static member using scope
resolution

int main() {
    Counter::count = 5; // Access using class name
    cout << "Count = " << Counter::count << endl;
    return 0;
}
```

Contd.

Example: Accessing Namespaces

```
#include<iostream>
namespace MyNamespace {
    int value = 50;
}

int main() {
    cout << MyNamespace::value << endl;  // 50
    return 0;
}
```

Memory Management Operators

- Can we allocate memory in function *fun* and can still access it after execution of function *fun* ends?
- Many times we may not want memory to vanish once function execution ends, and at the same time, we may also not want it to linger throughout the program execution
- **Can we (programmer) control the lifetime of allocated memory?**

✓ **Dynamic memory** makes it possible for programmer to **control *lifetime* and *size* of allocated memory at runtime**

✓ This is possible because **dynamic memory (heap)** is **not tied to function scope or lifetime** — unlike **local variables (stack)** which are destroyed when the function ends.

Following **4 library functions** declared in **stdlib.h** can be used to control dynamic memory in C and C++

- malloc
- calloc
- realloc
- free

Contd.

- malloc and calloc functions allocate memory **at runtime** (on **heap section**) and return pointer to it
- Memory allocated by one call to malloc/calloc is always contiguous.
- By **default** value stored at memory allocated by **malloc is garbage**
- By **default** value stored at memory allocated by **calloc is zero**

Contd.

- So far we have created an array whose size can be specified at runtime , But that can be done using **variable length array** too.

```
int size;
cin >> size;
int arr[size];
```

Then what is the **actual benefit of using dynamic memory**

- We can **change the size** of dynamic array during runtime **using *realloc* function**
  - While size of variable length array can not be changed once specified
- We can **control lifetime** of dynamic memory during runtime **using *free* function**
  - While lifetime of variable length array is tied to the block in which it is declared

Contd.

Memory management operators (new and delete)

- Like C language, **malloc, calloc, realloc** and **free** functions will still work
- New operators **new** and **delete** are added. Keyword **new** is used to allocate memory while **delete** is used for deallocating memory allocated using **new**.
- **new** and **delete** are *operators* (and *keywords*). A.k.a **Free store operators**
- Like **malloc, calloc** and **realloc**, lifetime of memory allocated using **new** is controlled by programmer
- No inbuilt feature for garbage collection like other OOP languages
- You can use **malloc** and **new** in the same program. But you cannot allocate an object with **malloc** and free it using **delete**. Nor can you allocate with **new** and delete with **free** or use **realloc** on an array allocated by new.
- Use **vector** if you ever need to **realloc** (**realloc** would still work for memory allocated using **malloc** and **calloc,** but it is discouraged in favour of **vector**)

Contd.

- New – to create objects of any type.
- Delete – to destroyed objects.

- **Int *p = new int** or **Int *p = new int(25)**
- **Int *p = new int[10]**

- **delete p;** // to destroyed object pointed to by p.
- **delete []p;** //to destroyed the entire array pointed to by p.

Contd.

- Example:-

```
#include <iostream>
using namespace std;

int main() {
    int* ptr = new int; // Allocates memory in heap for one integer and
gives its address to ptr

    *ptr = 10; // store value in dynamically allocated memory
    cout << "Value stored at ptr: " << *ptr << endl;

    delete ptr; // free the memory
    ptr = nullptr; // good practice to avoid dangling pointer

    return 0;
}
```

Contd.

- Example:-

```
#include <iostream>
using namespace std;

int* fun() {
    int* p = new int;  // dynamically allocate memory
    *p = 42;
    return p;          // return pointer to allocated memory
}

int main() {
    int* ptr = fun();  // call the function and get the pointer
    cout << "Value: " << *ptr << endl;

    delete ptr;        // free the memory after use
    ptr = nullptr;

    return 0;}
```

Contd.

But if,

```
#include <iostream>
using namespace std;

int* fun() {
    int x = 10;
    return &x;  // BAD! x is local, memory is gone after
function ends
}
```

- The variable x is stored on the stack.
- Once fun() finishes, x is destroyed.
- Returning &x gives us a dangling pointer — accessing it causes undefined behavior.

Type Cast Operator

- Traditional C casts are augmented in C++ by a function-call notation as a syntactic alternative. The following two versions are equivalent:
  - (type-name) expression // C notation
  - type-name (expression) // C++ notation

Example:

average = sum/(float)i; // C notation

average = sum/float(i); // C++ notation

But , **P = int * (q);** is illegal. In such cases, we must use C type notation **P = (int *) q**;

Alternatively, we can use **typedef**

```
typedef  int *  int_pt;
p =  int_pt (q);
```

Implicit Conversions

- We can mix data types in expressions. For example,

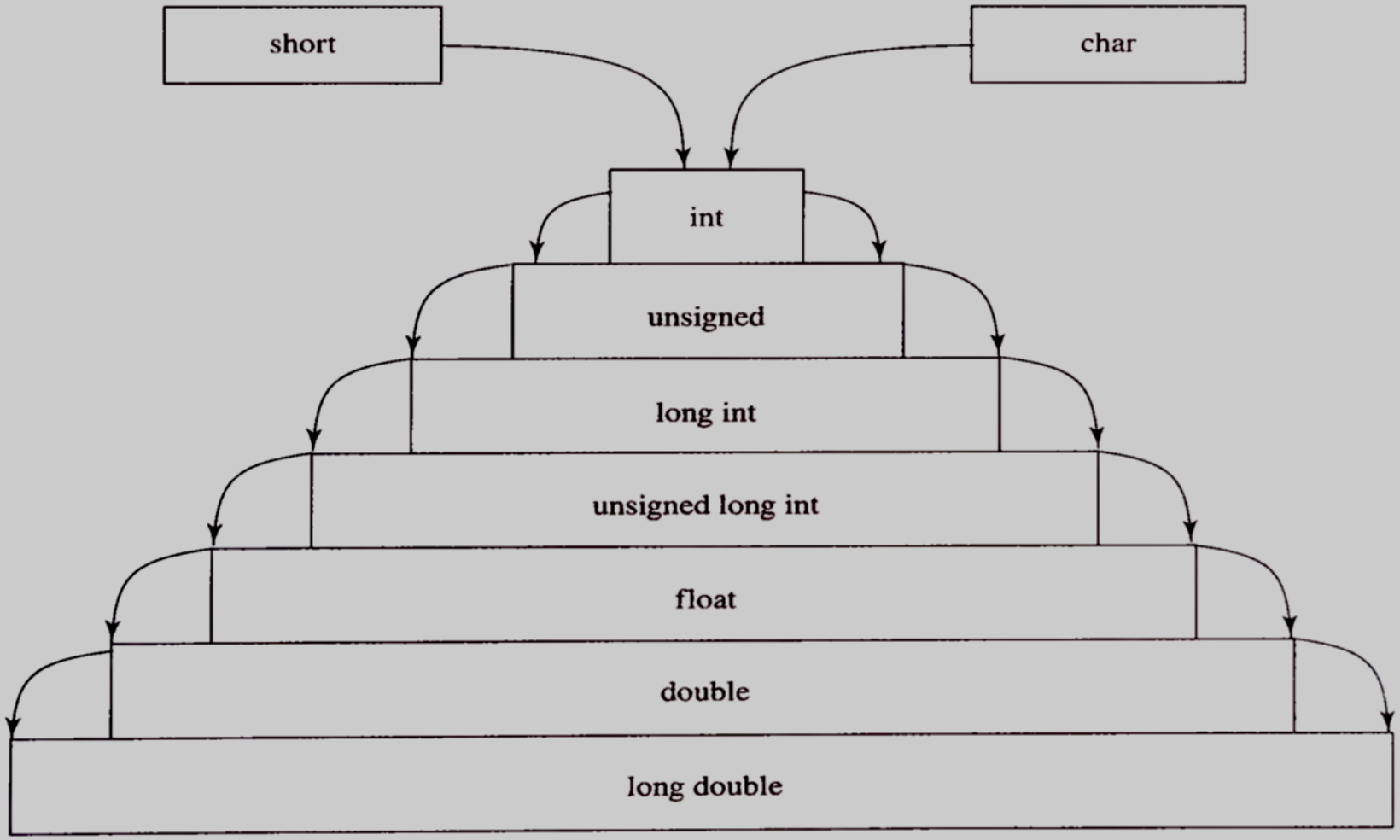
m = 9 + 3.69 is a valid statement.

Wherever data types are mixed in an expression, C++ performs the conversions automatically. This process is known as implicit or automatic conversion.

When the compiler encounters an expression, it divides the expressions into sub-expressions consisting of one operator and one or two operands.

For a binary operator, if the operands type differ, the compiler converts one of them to match with the other, using the rule that the **"smaller" type is converted to the "wider" type**. For example, if one of the operand is an **int** and the other is a **float**, the int is converted into a float because a float is wider than an int.

Contd.



Water-fall model of type conversion

Contd.

- Whenever a char or short int appears in an expression, it is converted to an int. This is called **integral widening conversion**. The implicit conversion is applied only after completing all integral widening conversions.

Results of mixed-mode operations

LHO \ RHO	*char*	*short*	*int*	*long*	*float*	*double*	*long double*
char	int	int	int	long	float	double	long double
short	int	int	int	long	float	double	long double
int	int	int	int	long	float	double	long double
long	long	long	long	long	float	double	long double
float	float	float	float	float	float	double	long double
double	double	double	double	double	double	double	long double
long double	long double	long double	long double	long double	long double	long double	long double

RHO – Right-hand operand
LHO – Left-hand operand

Special Assignment Expressions

- **Chained Assignment :**
  - x = (y = 10) ;

    or
  - x = y = 10

But , **float x = y = 12.34 ;** is wrong.

Correct way is float a = 12.34 , b = 12.34 ;

- **Embedded Assignment :**
  - x = (y = 36) + 9 ;

(y = 36) is an assignment expression known as **embedded assignment**. Here, the value **36** is assigned to y and then the result 36+9 = 45 is assigned to x. This statement is identical to

y = 39;

x = y + 9;

H.W

<u>Problem</u> :- Write a C++ program to input marks of n students using dynamic memory allocation. Use new and delete[] for managing memory. Use a global variable count to track the number of students processed, and access it in main() using the scope resolution operator. Calculate total and average marks using a compound assignment (e.g., total += marks[i];) and calculate average using type casting from int to float. Also demonstrate a special assignment expression.

H.W

<u>Problem</u> :- An electricity board charges the following rates to domestic users to discourage large consumption of energy:

For the first 100 units - 60P per unit

For next 200 units - 80P per unit

Beyond 300 units - 90P per unit

All users are charged a minimum of Rs. 50.00. If the total amount is more than

Rs. 300.00 then an additional surcharge of 15% is added. Write a program to read the names of users and number of units consumed and print out the charges with names.